



LISTA 12 - OPERAÇÃO BIG DATA

(Lambdas e Streams API)

A Sua Missão: Você foi contratado pela *TechNexus*, um e-commerce global. O antigo programador foi demitido porque os relatórios da loja demoravam 4 horas para rodar (ele usava dezenas de laços `for` e `if` pesados). Sua missão é assumir a Engenharia de Dados e reescrever os relatórios usando o poder atômico do **Java Streams** e das **Funções Lambda**, resolvendo os problemas em apenas uma linha de código fluente.



SETUP: A FORJA DA IDE

Siga o passo a passo exato para evitar erros fantasmas do VS Code fazendo a criação correta.

1. Feche o VS Code atual e abra uma janela limpa.
2. Aperte F1 (ou Ctrl + Shift + P) no VS Code.
3. Digite Java: Create Java Project e aperte Enter.
4. Escolha No build tools, selecione a sua Área de Trabalho e chame o projeto de `missao-08-streams`.
5. Apague o arquivo `App.java`.
6. No menu inferior esquerdo (**JAVA PROJECTS**), passe o mouse sobre a pasta `src`, clique em + (New Java Class) e crie os pacotes e as classes a seguir (ex: `br.com.technexus.model.Produto`).



NÍVEL 1: A BASE DE DADOS

Antes da mágica acontecer, precisamos do molde dos nossos dados.

Exercício 1: A Classe Produto

- ☐ Crie a classe `Produto` no pacote `br.com.technexus.model`.
- ☐ Crie os atributos `private`: `nome (String)`, `categoria (String)` e `preco (double)`.
- ☐ Use o *Source Action* (botão direito) do VS Code para gerar o Construtor (com os 3 parâmetros) e os métodos Getters.
- ☐ (Opcional, mas recomendado) Gere o método `toString()` para imprimir o produto de forma bonita no console depois.

Exercício 2: A Loja

- ☐ Crie a classe `Loja`.
- ☐ Dentro dela, declare e instancie um "banco de dados" em memória:

```
private List<Produto> catalogo = new ArrayList<>();
```
- ☐ Crie um método tradicional para colocar itens nessa lista:

```
public void cadastrar(Produto p) { this.catalogo.add(p); }
```

🟡 NÍVEL 2: A ESTEIRA ROBÓTICA (Streams API)

💡 DICA DE SINTAXE: Peneirando Dados (Filter)

```
public List<Objeto> filtrarAlgo() {  
    return minhaLista.stream()  
        .filter(x -> x.getPropriedade() == alvo)  
        .toList(); // Ou .collect(Collectors.toList())  
}
```

Exercício 3: O Relatório de Categoria

O departamento de marketing quer ver apenas a prateleira de "GAMES".

- ☐ Na classe Loja, crie o método: `public List<Produto> buscarPorCategoria(String catDesejada).`

Regra do Chefe: É proibido usar `for` e `if`.

Sua Tarefa: Usando o `return this.catalogo.stream()...`, coloque a "peneira" (`filter`) para deixar passar apenas os produtos onde `p.getCategoria().equalsIgnoreCase(catDesejada)`. Lembre-se de colocar a caixa no final da esteira (`.toList()`).



DICA DE SINTAXE: Extraindo e Calculando (`MapToDouble` e `Sum`)

```
public double somarDinheiro() {  
    return minhaLista.stream()  
        .mapToDouble(x -> x.getDinheiro())  
        .sum();  
}
```

Exercício 4: O Fechamento de Caixa

A contabilidade precisa saber qual é o valor total financeiro em estoque.

- ☐ Na classe Loja, crie o método: `public double calcularPatrimonioTotal().`

Sua Tarefa: Pegue o catálogo, transforme em `.stream()`, use o robô transformer `.mapToDouble(p -> p.getPreco())` (ou o método por referência `Produto::getPreco`) e esprema o resultado financeiro final com o `.sum()`. Retorne esse valor.

● NÍVEL 3: COMPONDO A LINHA DE MONTAGEM (Pipeline)

Exercício 5: O Filtro Financeiro Avançado

O chefe enlouqueceu e juntou os dois relatórios em um só: *"Quero saber o valor somado APENAS dos livros!"*

☐ Na classe Loja, crie o método: `public double calcularTotalPorCategoria(String catDesejada).`

Sua Tarefa: Enfileire os robôs na mesma esteira! Primeiro o `.filter()` para deixar passar apenas a categoria desejada. DEPOIS o `.mapToDouble()` para extrair o preço. POR ÚLTIMO o `.sum()` para o resultado final.

● O CAMPO DE PROVAS (A Classe Main)

Exercício 6: Testando o Código Fluente

- ☐ Crie a classe Main (pacote `br.com.technexus.main`) com o `public static void main`.
- ☐ Instancie a sua Loja.
- ☐ Cadastre 5 produtos variados. Ex:
 - ☐ 2 "GAMES" (Ex: The Witcher a R\$ 150, FIFA a R\$ 200).
 - ☐ 2 "LIVROS" (Ex: Java for Dummies a R\$ 100, Clean Code a R\$ 80).
 - ☐ 1 "HARDWARE" (Ex: Mouse a R\$ 50).
- ☐ Imprima no console o resultado do método `buscarPorCategoria("GAMES")`. O console não pode exibir o Mouse nem os Livros!
- ☐ Imprima no console o resultado do método `calcularPatrimonioTotal()`. O número total de todo o estoque deve aparecer.
- ☐ Imprima no console o resultado do `calcularTotalPorCategoria("LIVROS")`. O valor esperado deve ser a soma exata dos seus livros.



O seu código processou milhões de bytes em $O(n)$ otimizado internamente com apenas 3 linhas de código fluente. Dê o git commit e push da sua vitória!



DESAFIO DE ENGENHARIA / SEGURANÇA (APENAS PARA OS FORTES)

A TechNexus foi invadida na noite passada por um ex-funcionário.

Ele criptografou o link de acesso ao servidor de banco de dados oficial do E-commerce e deixou um bilhete zombando da sua capacidade de quebrar senhas usando Java.

Você não tem a senha, mas tem as dicas de infraestrutura e o processamento de uma linguagem poderosa nas mãos.

Os Dados Capturados no Servidor:

- **Link Cifrado (Aberto apenas via Terminal UNIX/Linux ou Java Runtime):**

```
U2FsdGVkX189CvKETNa+k2wHIMpbCwNk7HKB3nBRzOD9bBaPt2nFMCdElvKoRfTmmqVv41Trh37ORXFWRVNOX3  
vpgPHULkkaoyh9DfmzrGBXkGnu/SJfQkGuU08zbgMQNSwCGTwoIHkUMzRFQELN0Q==
```

- **Algoritmo:** `aes-256-cbc` com `-pbkdf2` e `-iter 1000`.
- **O Padrão da Senha:** Exatamente **6 caracteres**, contendo apenas letras minúsculas e/ou números.
- **Dica de Inteligência (Vazamento do Log):** A senha **obrigatoriamente** começa com as letras `lam`. (Portanto, você só precisa adivinhar os 3 últimos caracteres!).

A Sua Missão HACKER:

Construa um código em Java (uma nova classe `ForcaBruta`) que execute a força bruta (Brute Force) de forma inteligente.

1. Gere todas as combinações possíveis para os 3 caracteres restantes (de `lam000` até `lamzzz`).
2. Use o pacote de criptografia do Java (`javax.crypto.Cipher`) ou faça o Java disparar um comando no terminal do sistema operacional (usando `Runtime.getRuntime().exec()`) para testar a senha contra o arquivo/link cifrado usando a biblioteca OpenSSL nativa.
3. Se a senha abrir o cofre, o seu código deve interromper a busca e estampar a URL secreta no console!

Que comecem os jogos. 🐞